Project Continuation Report: From Profiling Existing Bayesian Optimization Frameworks to Designing a Parallel Multi-Surrogate Prototype

Meron Oumer
MPR Lab, Smith College

End-of-Semester Handoff Report

Abstract

This report documents the full arc of a semester-long research effort on Bayesian Optimization (BO) in high-performance computing (HPC) environments. The project began as a systems-level profiling study of widely used BO frameworks and baseline search methods across synthetic functions and machine learning workloads. That baseline study established a reproducible empirical foundation by decomposing runtime into Gaussian Process (GP) fitting, acquisition optimization, and objective evaluation costs. The resulting analysis showed that evaluation time dominates expensive machine learning tasks, while GP fitting and acquisition overhead become severe bottlenecks in low-cost or GP-heavy settings. These findings motivated a shift toward framework design.

The second phase of the project focused on identifying a technically meaningful next step for a new optimization system. Multiple directions were considered, including low-level implementations in Rust, asynchronous multi-fidelity BO, and non-GP surrogates. A literature review on multi-fidelity optimization, learning-curve methods, constrained BO, and surrogate alternatives suggested that while multi-fidelity methods are highly effective for reducing evaluation waste, many existing systems remain GP-heavy, rely on strong fidelity assumptions, or are sequential in structure. Feedback from Professor Michael further pushed the project away from a simple two-stage sequential design and toward a more explicitly parallel, HPC-centered approach.

The project therefore converged on a prototype direction centered on a parallel two-surrogate optimization framework. In this design, two surrogate models share a common optimization history, generate proposals in the same round, and have their suggestions merged into one evaluation batch. The goal is not to assume the best surrogate pair in advance, but to build a framework that can compare pairings systematically. As of the end of the semester, the architectural skeleton of this framework has been designed in detail and not yet implemented, including a shared datastore, surrogate interface, candidate merge logic, worker-pool evaluation layer, orchestrator, and experiment harness. However, the prototype is not yet fully validated end-to-end across all intended surrogate pairs. This report is intended as both a technical summary and a handoff document so that future work can continue from a clear, well-documented state.

1. Executive Summary

The project started with a concrete systems question: *where does the time actually go inside modern Bayesian Optimization frameworks when they are used on synthetic functions, classical machine learning models, and large neural workloads in HPC-like settings?* The initial answer came from a baseline profiling study that instrumented major BO frameworks and search baselines across a diverse workload suite. That study found three main patterns. First, expensive model training usually dominates runtime on realistic ML tasks. Second, GP fitting and acquisition optimization remain serious bottlenecks in GP-based frameworks, especially on synthetic functions and low-cost settings. Third, DeepHyper-like lightweight asynchronous systems incur much less internal BO overhead than BoTorch- or Optuna-style GP-heavy frameworks [5].

Those findings shaped the second stage of the project: rather than proposing an optimization framework in the abstract, the new framework direction should target bottlenecks that were actually measured. Several ideas were explored in sequence, including a low-level Rust implementation, multi-fidelity BO, adaptive refresh logic, and multi-surrogate systems. The literature review on multi-fidelity optimization showed that reducing evaluation cost is a very important direction, but also revealed two challenges for the current project: many strong multi-fidelity methods rely on GP-based modeling or strong assumptions about learning-curve predictiveness, and several plausible designs were already represented in sequential or hierarchy-based forms [11, 2, 6, 12]. Advisor feedback ultimately made the decisive shift: rather than spending the limited remaining time building a sequential system or overcommitting to one assumed surrogate pairing, the project should build a framework that can compare surrogate pairings directly.

The project therefore ended the semester with a clear prototype direction: a **parallel two-surrogate Bayesian optimization framework** with a shared datastore, a common optimization loop, candidate merging, and per-surrogate contribution logging. The current implementation state is a *framework skeleton plus partial prototype*. This report captures both the completed empirical work and the unfinished design/implementation state.

2. Project Motivation and Scope

Bayesian Optimization is one of the standard approaches for optimizing expensive black-box objectives, especially when each trial requires meaningful computation, such as model training, simulation, or system tuning [1, 8]. In principle, BO improves efficiency by replacing brute-force search with a surrogate model and an acquisition strategy that choose promising candidates intelligently. In practice, however, BO is itself a system with real runtime costs. A BO framework must maintain a surrogate model, optimize or evaluate an acquisition strategy, coordinate trials, and often interact with external scheduling or evaluation infrastructure. In HPC contexts, these internal costs matter.

The original motivation for this project was therefore not only to ask whether BO converges faster in terms of sample efficiency, but also whether it remains computationally worthwhile after accounting for its own systems overhead. This distinction is especially important in settings where:

- objective evaluations are relatively cheap, making internal BO overhead more visible,
- evaluations are parallelized, so sequential BO components can idle otherwise available workers,

- workloads span a large range of dimensionalities and evaluation costs, and
- practitioners must choose between sophisticated model-based methods and simpler baselines such as random or grid search.

The project therefore had two linked goals:

1. build a rigorous baseline understanding of the runtime behavior of existing BO frameworks; and
2. use that evidence to motivate a better framework design]

3. Baseline Profiling Study

3.1 Goal of the baseline study

The first phase of the project produced a framework-agnostic profiling study of BO systems across synthetic and ML tasks. The purpose of that study was to isolate and measure the computational cost of individual BO components: surrogate fitting, acquisition optimization, and objective evaluation [5]. Rather than evaluating frameworks only by convergence or final score, the study measured what each framework was spending time on.

3.2 Frameworks and tasks profiled

The baseline study evaluated:

- **BoTorch**, as a modular research-oriented GP-heavy BO framework,
- **DeepHyper**, as an asynchronous HPC-oriented framework emphasizing large-scale parallelism,
- **Optuna**, as a practical black-box optimization framework with pruning and distributed support,
- **Ray Tune** where available in the original profiling setup,
- **Grid Search**, and
- **Random Search**

across synthetic benchmark functions and ML workloads [5].

The workload suite included:

- synthetic functions: Branin, Hartmann6, Rastrigin,
- classical ML models: logistic regression, random forests, XGBoost,
- large pretrained models: DistilBERT and T5-Large

in the baseline study's most complete form [5].

3.3 Metrics collected

The profiling instrumentation focused on:

- GP fit time,
- acquisition optimization time,
- evaluation time,
- total iteration time,
- and time-to-quality behavior over cumulative wall-clock time.

This design was motivated by a gap in the BO literature: many studies report optimization quality or convergence, but do not decompose where time is spent inside the BO system itself [5].

3.4 Main empirical findings

Several findings from the baseline study shaped everything that followed.

Finding 1: evaluation time dominates realistic ML tasks. For classical ML and large neural workloads, model evaluation (that is, training or fine-tuning the task model) was the dominant cost component. This means that for expensive tasks, reducing the number of wasted evaluations remains crucial [5].

Finding 2: GP fitting and acquisition overhead can dominate low-cost tasks. On synthetic benchmarks and other cheap objectives, BoTorch and Optuna showed severe relative overhead from GP fitting and acquisition optimization. The study notes that even simple tasks such as Branin could produce iteration times above two seconds in BoTorch, exposing GP-heavy framework cost in low-latency settings [5].

Finding 3: DeepHyper's internal overhead was consistently low. DeepHyper maintained near-zero surrogate fit time and sub-millisecond candidate-selection overhead across a wide range of tasks, including very expensive workloads. This suggested that lightweight surrogate or scheduling approaches may be especially promising for HPC contexts [5].

Finding 4: GP-based frameworks scale poorly as observations grow. The baseline study repeatedly observed GP fitting cost growth with increasing dimensionality, larger observation sets, and longer runs, reinforcing the well-known scalability limitations of standard GP modeling [5, 10].

Finding 5: simple baselines can be surprisingly competitive. Grid and random search performed very well for low-cost synthetic tasks because they have almost no internal systems overhead. This helped sharpen the project's central question: a more complicated BO framework is only worthwhile if its added intelligence clearly compensates for its own runtime cost [5].

3.5 Why this baseline study mattered

The baseline work did more than produce plots. It narrowed the design space. It ruled out naive assumptions such as:

- "just use GP-based BO and parallelize it later,"
- "the only thing that matters is optimization quality,"
- "DeepHyper-like distributed systems automatically solve the hard part,"
- or "framework overhead is negligible compared to evaluations in all cases."

Instead, it suggested a more careful direction: any new system should be designed with explicit attention to both *evaluation-side bottlenecks* and *surrogate-side bottlenecks*.

4. Literature Review That Informed the Design Shift

4.1 What the literature review was trying to answer

After the profiling phase, the project shifted into literature review with a design question in mind: *if existing BO frameworks exhibit costly GP fitting, acquisition overhead, and distributed inefficiencies, what kind of new framework would actually address those weaknesses?*

The literature review therefore did not focus only on generic BO theory. It focused on four more specific questions:

1. How do current systems reduce *evaluation cost*?
2. How do they reduce *surrogate modeling cost*?
3. Have multi-surrogate or multi-fidelity designs already been explored, and in what form?
4. Where is there still room for a systems-level contribution?

4.2 Why multi-fidelity became interesting

A major theme across the literature was that the time required to evaluate a configuration is often the dominant cost in HPO and related optimization settings. Multi-fidelity methods address this by using cheaper approximations of expensive evaluations: fewer epochs, smaller datasets, lower-resolution simulations, or partially trained models [11, 2, 12].

The appeal was immediate for this project because the baseline profiling study also showed that evaluation time dominates realistic ML workloads. If the largest cost is evaluation, then a strong new framework should eventually include some way of reducing evaluation waste.

4.3 Important multi-fidelity papers and what they contributed

The literature review covered several important papers and systems.

Won et al. (2025): a review on multi-fidelity HPO. This review was useful because it organized the design space of multi-fidelity HPO, including Hyperband-style resource allocation, BOHB-like combinations of BO and successive halving, and systems-level concerns around synchronous waiting and wasted time [11]. One important takeaway was that some multi-fidelity approaches still underuse low-fidelity information or suffer from implementation complexity when parallelized.

Do and Zhang (2024): review of multi-fidelity BO. This review broadened the discussion beyond HPO into engineering optimization, emphasizing the efficiency–accuracy tradeoff when combining low- and high-fidelity models [2]. It also helped clarify that many multi-fidelity systems rely on strong assumptions about correlation between cheap and expensive fidelities.

Rakotoarison et al. (2024): in-context freeze-thaw BO. This work was especially important because it attacked a different bottleneck: surrogate-update overhead. Instead of repeatedly retraining a learning-curve model online, it used in-context learning to make freeze-thaw style decisions in a single forward pass [6]. This reinforced the idea that *internal BO computation cost itself* can be a valid systems target, not just evaluation cost.

Wu et al. (2020): practical multi-fidelity BO for HPO (taKG). This work showed how trace observations and warm-starting can reuse partial training and avoid restarting from scratch [12]. It supported the idea that systems should preserve and reuse work rather than treat every evaluation as completely independent.

Foumani et al. (2025) and Wang et al. (2025): constrained cost-aware multi-fidelity BO. These papers helped frame multi-fidelity selection as a cost-aware decision problem rather than just a fixed staging scheme [3, 9]. This influenced later project discussions about whether surrogate updates or fidelity progression should be fixed in advance or triggered more adaptively.

Koziel et al. (2025): expedited antenna optimization. This paper was useful as an example of a practical two-stage sequential strategy: broad search at low fidelity followed by focused high-fidelity tuning [4]. It became important later because it showed that simple sequential hierarchies are effective, but also suggested that such designs may be too close to known patterns to serve as the strongest contribution for this project.

Rao and Jaganathan (2023): adaptive Bayesian contextual Hyperband. This paper reinforced the value of adaptive resource allocation and dynamic thresholds [7], though it also illustrated the tradeoff between exploiting promising candidates early and possibly missing late-improving ones.

Zhuo et al. (2025): GP-based multi-fidelity BO for calibration. This paper was representative of a large class of GP-based multi-fidelity systems [13]. It was useful as a reminder that multi-fidelity methods often remain computationally dependent on GP surrogates, which was not ideal given the profiling results.

4.4 What the literature suggested about non-GP and alternative surrogates

A second important theme in the literature was the tension between expressive uncertainty-aware models and computational cost. Standard GPs remain attractive because of their uncertainty estimates, but scale poorly in observation count and can become a systems bottleneck [8, 10]. Alternatives such as Bayesian neural networks, Student-t processes, or learned surrogates have been explored, but often add their own implementation complexity or tuning burden [5].

For this project, the practical implication was not that GPs should be discarded entirely. Rather, it suggested that a framework with only one expensive surrogate might be too brittle, while a framework that can mix lightweight and more expressive surrogates could better balance cost and search quality.

5. Research Directions Considered and Why the Project Pivoted

5.1 Direction 1: low-level systems implementation (Rust / non-Python BO)

One early reaction to the profiling results was to consider a lower-level implementation, possibly in Rust or C/C++, to reduce Python communication and orchestration overhead. This idea was attractive from an HPC and systems perspective: if Python itself contributes friction in distributed BO workflows, then a lower-overhead language might help. However, this risks shifting the work away from BO design into framework engineering without enough time to produce a convincing research result.

5.2 Direction 2: multi-fidelity BO as the main prototype

The next serious direction was asynchronous or parallel multi-fidelity BO, motivated by the fact that evaluation time dominates realistic tasks. This direction was technically strong and remains an important future extension. However, two concerns emerged:

1. the literature already contains many strong multi-fidelity systems, including sequential, constrained, review-based, and learning-curve variants; and
2. several plausible initial designs the project considered were too close to existing sequential or stage-based patterns.

In addition, Professor Michael's feedback emphasized that a contribution built around fixed update schedules or simple two-stage movement between models risked being either too arbitrary or too close to known designs.

5.3 Direction 3: sequential two-surrogate BO

Another direction considered was a two-stage or sequential multi-surrogate system, where one model would explore or filter candidates and another model would refine them. This was conceptually natural, especially after reading sequential or staged multi-fidelity systems. However, it ran into two problems:

- sequential multi-model flows are easier to find analogs for in existing literature, and

- from an HPC perspective, a strictly sequential design underuses available parallelism.

Professor Michael's feedback was especially important here: the project should not just combine two models, but should do so in a way that actually takes advantage of the HPC setting.

5.4 Direction 4: fixed-cost GP refresh with a fast surrogate

A later iteration of the idea proposed a fast surrogate retrained every round and a slower GP surrogate refreshed every K rounds. This was directly motivated by the profiling study: if GP fitting is expensive, amortizing it across multiple rounds seems sensible. This design was good enough to clarify the systems question, but advisor feedback exposed its weakness: a fixed cadence such as "refresh the GP every 5 rounds" is ultimately arbitrary. It answers *how often* without first answering *when it is worth paying for a refresh.* That criticism pushed the project toward a broader and more flexible design.

5.5 Final pivot: framework-first pair testing

The project therefore landed on a cleaner and more defensible direction:

> **Build a lightweight parallel two-surrogate BO framework first, then use that framework to compare different surrogate pairings rather than assuming the best pair in advance.**

This pivot solved several problems at once:

- it made the contribution more framework- and systems-oriented,
- it aligned with HPC by keeping both surrogates active in the same round,
- it avoided overcommitting too early to one fixed refresh logic or one surrogate pair,
- and it turned the next phase into a controlled experimental question: *which surrogate pair works best together under the same shared optimization loop?*

6. Final Prototype Direction

6.1 Core idea

The final prototype direction at the end of the semester is a **parallel two-surrogate Bayesian optimization framework** with the following high-level behavior:

1. maintain one shared optimization history,
2. fit two surrogate models on that same history,
3. let both surrogates generate candidate points in the same round,
4. merge those proposal streams into one final evaluation batch,
5. evaluate the batch in parallel,

6. append results back into the shared datastore,

7. repeat.

This framework is deliberately designed to support *pair testing*. Rather than hardwiring the final answer as "GP + RF" from the start, the architecture is supposed to allow testing surrogate combinations such as:

- GP + RF,
- GP + TPE,
- RF + TPE,

with the understanding that some of these are not yet fully implemented.

6.2 Why this direction makes sense

This design was chosen for three main reasons.

Reason 1: it is directly grounded in the baseline profiling study. The profiling results showed that GP-heavy frameworks can incur substantial internal overhead, while lightweight systems can keep internal BO costs small [5]. A multi-surrogate framework lets the project compare expensive and lightweight models under one shared loop.

Reason 2: it is more HPC-centered than a sequential design. Two surrogates working in the same round is more consistent with the project's HPC framing than a purely sequential multi-model pipeline. The goal is not just to have two models, but to have a system architecture that keeps candidate generation and evaluation organized for parallel use.

Reason 3: it separates framework design from model choice. This is perhaps the cleanest conceptual improvement in the project. Instead of deciding the "correct" pair too early, the project now treats surrogate pairing as an experimental question supported by the framework itself.

7. Current Prototype Architecture

7.1 Four main layers

The prototype currently centers on four main conceptual components:

1. DataStore The datastore is the shared optimization history. It stores every evaluated candidate as a structured record, including the candidate vector, objective value, round identifier, surrogate source, timing fields, and later contribution-logging metadata.

2. Surrogate layer Two surrogate models are fit on the same history. Each surrogate implements the same core interface: fit on observed data, predict on candidate points, and propose a ranked list of candidate configurations.

3. Candidate merge + worker pool Candidate proposals from the two surrogates are merged into one batch. That batch is then evaluated by the worker pool, which runs the objective on each candidate and returns results along with timing information.

4. Orchestrator The orchestrator is the control loop that ties the system together. It handles warm-start initialization, surrogate fitting, proposal generation, candidate merging, batch evaluation, dataset updates, and metrics logging.

7.2 Current architecture flow

The conceptual round-level flow is:

1. read all observations from the datastore,
2. fit surrogate A and surrogate B,
3. request proposals from both,
4. merge proposals into one batch,
5. evaluate the batch,
6. append results back into the datastore,
7. update run metrics and continue.

This is still synchronous in the current design. A full round completes before the next round begins.

7.3 Per-surrogate contribution logging

A very important addition in the later prototype design was per-surrogate contribution logging. Each evaluated candidate is intended to store:

- `surrogate_source`: which surrogate proposed it,
- `candidate_rank_within_source`: its rank in that surrogate's proposal list,
- `improved_global_best`: whether it improved the best observed objective value at the time it was inserted.

This is essential because the framework is not only trying to optimize well overall; it is also trying to answer *which surrogate contributed meaningfully within a pair.*

7.4 Important current limitations

TPE is not yet implemented. This is the main reason why the pair-testing framework is not yet experimentally complete. The harness is designed to skip unsupported TPE-based pairs honestly rather than pretending they run.

Some internal naming still reflects the first GP+RF use case. Although the harness is pair-oriented, parts of the underlying orchestration and metrics logic still use GP/RF-specific labels. This is not fatal, but it is a cleanup task if the framework is to support arbitrary surrogate pairs cleanly.

Objective-direction consistency must remain explicit. The framework currently follows a "higher is better" convention, which required a sign correction for the logistic regression objective. Future contributors should preserve that convention consistently across all added tasks.

8. What the Literature and Experiments Suggest About the Prototype

8.1 Why not just use one surrogate?

The literature and the profiling results together suggest that no single surrogate is obviously ideal for all cases:

- GPs offer good uncertainty modeling but can be expensive to fit,
- tree-based methods are much cheaper but may offer less principled uncertainty,
- practical HPO systems often benefit from cheap heuristics and scheduling rather than costly global modeling.

A multi-surrogate framework is therefore a reasonable attempt to balance these tradeoffs.

8.2 Why not stay with a simple sequential design?

The project explicitly moved away from sequential two-stage designs for two reasons:

1. they are less HPC-centered because one stage waits on the other, and
2. they are closer to ideas already represented in the literature, especially in staged multi-fidelity systems [4, 11].

8.3 Why multi-fidelity is still important, but deferred

The literature review strongly supports multi-fidelity methods as a future direction. Evaluation cost remains the dominant bottleneck in expensive ML tasks, and the ability to stop, continue, or cheaply screen poor configurations is likely essential in any future high-impact version of the framework [11, 2, 6, 12]. However, the project concluded that this should be layered *after* a clean two-surrogate framework exists, not built simultaneously in the first prototype.

9. Recommended Next Steps for Continuation

9.1 Immediate next steps

The most logical continuation plan is:

1. **Validate the GP+RF path end-to-end.** Run a small benchmark such as Branin with one seed and confirm that the new experiment harness, orchestrator, worker pool, datastore, and result-saving path work together correctly.

2. **Complete the missing TPE wrapper or choose a substitute third surrogate.** The framework is architected for pair comparisons, but currently only one pair is truly supported. The next concrete coding task is to implement the missing third surrogate family.

3. **Run the initial comparison matrix.** Once at least two pairings are runnable, run the planned matrix on:

   - Branin,
   - Hartmann,
   - Rastrigin,
   - logistic regression

   over seeds 42, 123, and 456.

4. **Analyze pair-level and surrogate-level contribution metrics.** Do not only compare final best objective values. Compare:

   - best objective versus wall-clock time,
   - fit-time contribution by model,
   - how often each surrogate improves the current best,
   - and whether one surrogate dominates the other inside a pair.

9.2 Second-stage next steps

After the pair-testing framework is stable, the project should likely move toward one of these directions:

Direction A: stronger multi-surrogate scheduling Once multiple pairings run correctly, the next systems question becomes whether candidate splits, surrogate refreshes, or model usage should remain fixed or become adaptive.

Direction B: evaluation-side control The project should return to evaluation cost explicitly through:

- pause/resume designs,
- early stopping,
- asynchronous scheduling,
- or a multi-fidelity extension.

Direction C: lower-overhead implementation or systems optimization If the framework demonstrates promise, a later phase could revisit lower-level implementation, cheaper communication, or compile-time/runtime systems optimizations.

10. Practical Handoff Notes

10.1 What a future contributor should read first

A future contributor should start with:

1. this continuation report,
2. the baseline profiling write-up [5],
3. the experiment harness and orchestrator,
4. the surrogate base class and the RF/GP wrappers,
5. then the pair-testing experiment plan.

10.2 Where future contributors should be cautious

Future work should be careful about:

- accidentally mixing maximize/minimize objective conventions,
- assuming unsupported surrogate pairs are implemented,
- overfitting the framework to GP+RF-only naming and metrics,
- and adding too many new design ideas at once before validating the existing skeleton.

11. Conclusion

This project made real progress in two distinct but connected forms. First, it produced a strong empirical baseline: a systems-level profiling study showing how runtime is distributed inside BO frameworks across synthetic and ML workloads. That study established the practical importance of both evaluation cost and internal BO overhead, and showed that frameworks differ dramatically in how those costs appear [5]. Second, it used that empirical evidence to converge on a more thoughtful framework-design direction rather than jumping straight into implementation based on intuition.

The project did not finish with a polished final optimization system. Instead, it finished with something that is arguably more useful for continuation: a clear understanding of why certain directions are stronger than others, a literature-grounded justification for moving away from simple sequential designs, and a partially implemented framework architecture that supports direct comparison of surrogate pairings in a shared optimization loop.

The intended continuation is therefore not ambiguous. The next milestone is to validate the current GP+RF path, implement the missing third surrogate needed for meaningful pair comparisons, and then use the new harness to compare pairings systematically. After that, the project can

return to the larger systems question of how to combine surrogate-side efficiency with evaluation-side efficiency. In that sense, the semester did not end with a dead end. It ended with a narrower, clearer, and more defensible research path.

References

[1] Eric Brochu, Vlad M. Cora, and Nando de Freitas. A tutorial on bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. *arXiv*, 2010.

[2] Bach Do and Ruda Zhang. Multi-fidelity bayesian optimization: A review, 2024. arXiv technical report.

[3] Zahra Zanjani Foumani, Mehdi Shishehbor, Amin Yousefpour, and Ramin Bostanabad. Constrained cost-aware multi-fidelity bayesian optimization, 2025. arXiv technical report.

[4] Slawomir Koziel, Anna Pietrenko-Dabrowska, and Stanislaw Szczepanski. Expedited machine learning antenna optimization using dimensionality reduction, multi-fidelity em models, and rapid final tuning through principal directions. *IEEE Early Access*, 2025.

[5] Meron Oumer, Anjia Lu, and Nargiz Akhmetova. Profiling bayesian optimization frameworks for synthetic and ml tasks: A baseline study, 2026. Internal project write-up, High Performance Computing Lab, Smith College.

[6] Herilalaina Rakotoarison, Steven Adriaensen, Neeratyoy Mallik, Samir Garibov, Edward Bergman, and Frank Hutter. In-context freeze-thaw bayesian optimization for hyperparameter optimization. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[7] Lakshmi Priya Swaminatha Rao and Suresh Jaganathan. Adaptive bayesian contextual hyperband: A novel hyperparameter optimization approach. *IAES International Journal of Artificial Intelligence*, 2023.

[8] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 2016.

[9] Jingyi Wang, Nai-Yuan Chiang, Tucker Hartland, J. Luc Peterson, Jerome Solberg, and Cosmin G. Petra. A constrained multi-fidelity bayesian optimization method, 2025. arXiv technical report.

[10] Andrew G. Wilson and Hannes Nickisch. Kernel interpolation for scalable structured gaussian processes (kiss-gp). *ICML*, 2015.

[11] Jonghyeon Won, Hyun-Suk Lee, and Jang-Won Lee. A review on multi-fidelity hyperparameter optimization in machine learning. *ICT Express*, 2025.

[12] Jian Wu, Saul Toscano-Palmerin, Peter I. Frazier, and Andrew Gordon Wilson. Practical multi-fidelity bayesian optimization for hyperparameter tuning. In *Proceedings of the 36th International Conference on Machine Learning*, 2020.

[13] Hua Zhuo, Jungang Ma, Mei Yang, Yikun Zhao, Lifang Yao, Yan Xu, and Kun Yang. Gaussian process-based multi-fidelity bayesian optimization for optimal calibration point selection. *Sensors*, 2025.